# LEX

**Generador de analizadores léxicos**

# LEX. Generador de analizadores léxicos

1. Introducción a LEX
2. Expresiones regulares
3. Variables predefinidas
4. Acciones
5. Entornos de reconocimiento
6. Compilación

# 01. Introducción a Lex

# LEX

- Generador de programas C para análisis léxico
- Basado en **expresiones regulares** a las que es posible asociar código en C
- Genera un programa en C que interpreta el AFD

# LEX

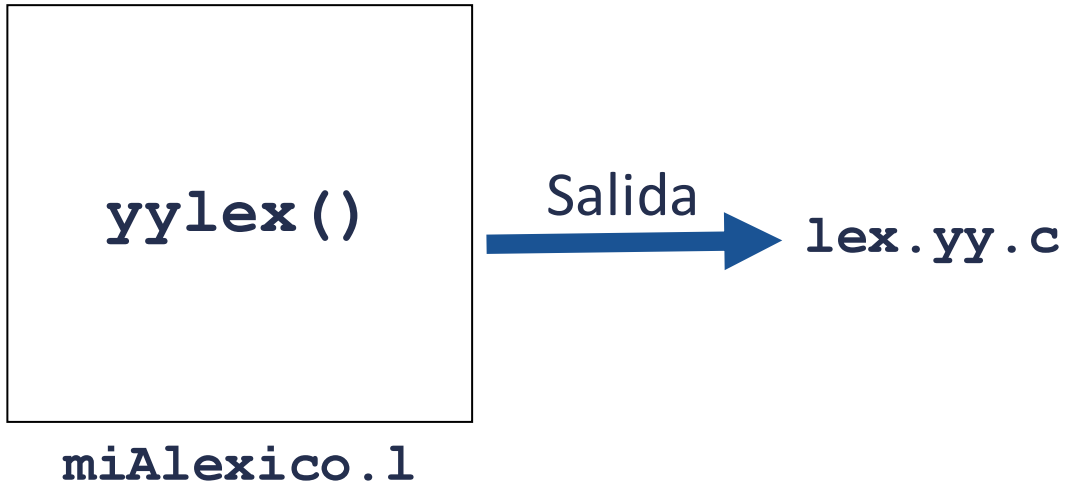
Un programa escrito en lex consta de **expresiones regulares** a las que se le asocian **acciones**

```
Expresión regular 1    {acciones (código C)}
Expresión regular 2    {acciones (código C)}
        …
```

**`miAlexico.l`**

# LEX

Para que el análisis comience es necesario llamar a la función **yylex()**



# Esquema de un fichero lex (.l)

```
 zona de definiciones
%%
   zona de reglas
%%
   zona de funciones del usuario. Código C
```

# Zona de definiciones

```
  Zona de definiciones
%%
  Zona de reglas
%%
  Zona de rutinas de usuario
```

- Declaraciones en código C (includes, defines, declaración de variables, prototipos de funciones). Se escribe entre

  **%{**

  **%}**

- Definiciones regulares (nombre definición). Se hace referencia a la definición usando **{**nombre**}**
- Definición de entornos de reconocimiento **%s**

# Ejemplo

Zona de definiciones
%%
Zona de reglas
%%
Zona de rutinas de usuario

Código C

```
%{
    #include<stdio.h>
    int contadorId = 0; // contador de identificadores

%}
digito (0|1|2|3|4|5|6|7|8|9)

letra (a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z)

id    {letra}({letra}|{digito})*

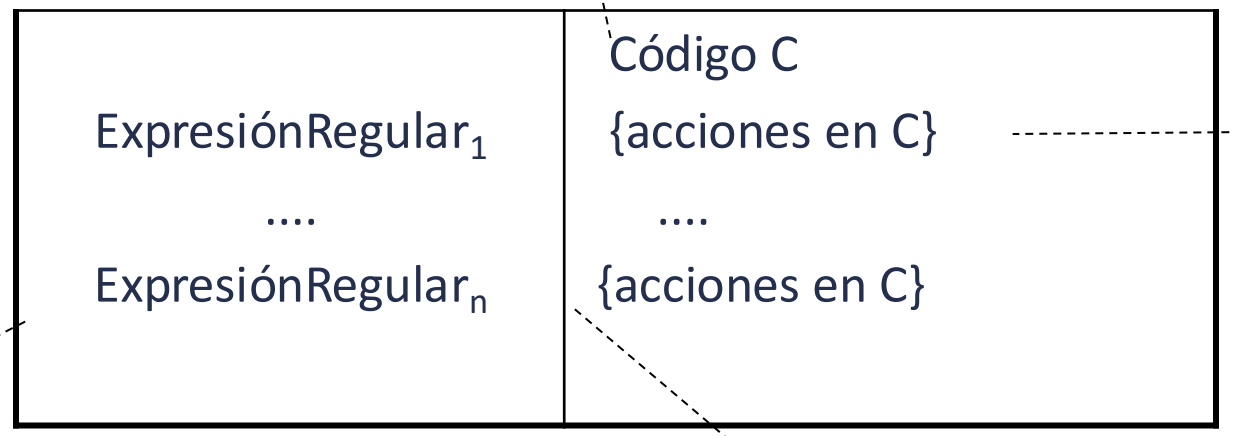
%%
```

Definiciones regulares

# Zona de reglas

Si se quiere que se ejecute un código antes de empezar a encajar en los patrones

Zona de definiciones
%%
Zona de reglas
%%
Zona de rutinas de usuario




Si sólo hay una acción se pueden omitir las llaves

IMPORTANTE
Debe empezar en la primera columna

Entre expresiónRegular y las acciones debe haber un espacio o un tabulador

# Zona de rutinas de usuario

```
  Zona de definiciones
%%
  Zona de reglas
%%
  Zona de rutinas de usuario
```

Contiene código C adicional necesario para la compilación. Normalmente la **función main** y la definición del resto de funciones

# Ejemplo

```
%{
      #include<stdio.h>
      int contadorId = 0;
%}
digito   (0|1|2|3|4|5|6|7|8|9)
letra    (a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z)

id       {letra}({letra}|{digito})*

%%
{id}    {contadorId++;
         printf("\n He encontrado un identificador");
        }

%%
main()
{
  yylex();
}
```

Función predefinida que desencadena todo el análisis

# lex.yy.c

```
(prototipos)
(código global)

int yylex(void) {
     (código local)
     (simulación del autómata)
}
(rutinas de usuario)
(tablas de transición del AFD)
```

# 02. Expresiones regulares

# Expresiones regulares en lex

- Como sabemos una expresión regular es un metalenguaje para describir lenguajes regulares
- Los caracteres del **código ASCII** son el **alfabeto** de entrada para las expresiones regulares
- **No existe** una forma de escribir la expresión regular **ε**

# Caracteres con significado especial

```
"   \   [   ]   ^   -   ?   .   *   +   |   (   )
$   /   {   }   %   <   >
```

Cuando se quiere introducir alguno de ellos en la expresión regular se utilizan los caracteres de escape " o \

ejemplo **"xyz++" ≡ xyz"++" ≡ xyz\+\+**

# Secuencias de escape

Lex también reconoce las secuencias de escape estándar

`\n` retorno de línea

`\t` tabulador

# Operadores

| | |
|---|---|
| . | cualquier carácter excepto el salto de línea<br>`a.` *reconoce las cadenas {aa,ab,ac,...a0,a1,…,a-,…}* |
| ? | Opcionalidad |
| * | 0 o más repeticiones |
| + | 1 o más repeticiones |

# Clases de caracteres. Alternativa

Delimitados por **[ ]**

Ejemplo **[abcd]** tiene como lenguaje asociado {a,b,c,d}

Dentro de una clase de caracteres **[ ], solo** hay tres caracteres con significado especial

**– ^ **

# Clases de caracteres

| | |
|---|---|
| - | Para especificar rangos. Puede formar parte de la clase si va el primero o el último |
| ^ | Si aparece debe ser el primero. Indica complementación, cualquier cosa que no sea |
| \ | Carácter de escape habitual |

Pierden su carácter especial ? . * + | ( ) $ / { } % < >

# Clases de caracteres. Ejemplos

| Exp. Regular | Lenguaje |
|---|---|
| `[0-9]` | {0,1,2,3,4,5,6,7,8,9} |
| `[-0-9]` | {-,0,1,2,3,4,5,6,7,8,9} |
| `[^a-z]` | Cualquier carácter que no sea una letra minúscula |
| `[^a-zA-Z]` | Cualquier carácter que no sea una letra (mayúscula o minúscula) |
| `[+-0-9]` | **ERROR. Expresión regular no válida** |
| `[-+0-9]` | {0,1,2,3,4,5,6,7,8,9,-,+} |

# Otros operadores

| | |
|---|---|
| **\|** | Alternancia |
| **( )** | Para agrupar |
| **{ }** | Utilizados para las definiciones regulares |
| **{n,m}** | Para especificar repeticiones de un número de veces.<br><br>Por ejemplo:<br>**a(b{1,3})** reconoce el lenguaje formado por**{ab,abb,abbb}** |

# Precedencia de operadores

***  ?  +** Son los operadores con mayor precedencia.

En caso de duda usa **paréntesis**

# Tratamiento de la ambigüedad

¿Es posible que lo que hay en la entrada encaje en más de un patrón?

## ¿Qué hace lex?

1. Elige siempre la entrada de mayor longitud posible
2. Si la longitud es igual elige la expresión regular escrita antes

# Tratamiento de la ambigüedad. Ejemplo

```
%%
        [a-z]+          {acciónLetras;}
        integer         {acciónPalabraReservada;}
%%
```

Si en la entrada tiene integers, ¿Qué acción haría? **acciónLetras**

Si en la entrada tiene integer, ¿Qué acción haría? **acciónLetras**

**IMPORTANTE**

Las expresiones regulares de palabras reservadas deben ir antes que la de identificadores

# Tratamiento de la ambigüedad. Ejemplo

```
%%
        integer       {acciónPalabraReservada;}
        [a-z]+        {acciónLetras;}

%%
```

Si en la entrada tiene integers, ¿Qué acción haría? **acciónLetras;**

Si en la entrada tiene integer, ¿Qué acción haría? **acciónPalabraReservada;**

# 03. Variables predefinidas

# Variables predefinidas por lex

| | |
|---|---|
| `yytext` | Lexema encajado<br>`char *` |
| `yyleng` | Longitud de yytext<br>`int` |
| `yyin` | Fichero de entrada<br>`FILE *` |
| `yyout` | Fichero de salida<br>`FILE *` |

```
%{     #include<stdio.h>  %}

pal    ([a-zA-Z]+)

%%
{pal}  {fprintf(yyout,"\n(%s, %d)",yytext,yyleng);}
.|\n   ;
%%

int main(){
       char nfe[20], nfs[20];

       printf("Entrada: "); scanf("%s",nfe);
       printf("Salida: "); scanf("%s",nfs);

       yyin = fopen(nfe, "r");
       yyout = fopen (nfs, "w");
       if (yyin != NULL && yyout!= NULL)
              yylex();
        else {printf("ERROR de apertura ");}
}
```

## ¿Qué hace?

Escribe en el fichero de salida una línea con cada palabra (compuestas sólo por letras) del fichero de entrada y su longitud

# 04. Acciones

# Acciones de lex

- Asociada a cada expresión regular hay una acción
- Cuando la entrada no encaja en ningún patrón (expresión regular), la acción por defecto es copiar la entrada a la salida.
- La acción nula se representa por **;**
- Cuando varias expresiones llevan asociadas la misma acción puede emplearse **|** para unir las líneas

# Acciones de lex

¿Qué debemos hacer si no queremos que se produzca ninguna acción?

Absorber la salida. Es decir plantear expresiones que se ajusten a todas las posibles entradas

# Acciones predefinidas en lex

**`int yywrap(void)`**

- Se llama cuando se alcanza el final del fichero
- En ella se puede cambiar el fichero al que apunta **yyin** y continuar con uno distinto
- Si devuelve 1 no sigue analizando nada más
- Lo habitual es ponerla en la zona de rutinas de usuario

```
int yywrap(){ return 1; }
```

# 05. Entornos de reconocimiento

# Entornos de reconocimiento

**`<E1,E2,...,En> expresiónRegular`**

- Sólo intentará encajar la expresión regular cuando nos encontremos en alguno de los entornos `E1,E2,...,En`
- El entorno predefinido es `INITIAL`. Las reglas de este entorno se pueden reconocer aunque nos encontremos en otro
- `E1,E2,...` y `En` son nombres arbitrarios formados por letras que deben ser declarados en la sección de definiciones, usando

  **`%s E1 E2 ..En`**
- `BEGIN(E);` Para cambiar de entorno, debe aparecer de forma aislada

# Ejemplo entornos de reconocimiento

```
%{
        #include<stdio.h>
%}

%s COMMENT CODE
%%
                {BEGIN(CODE);}
<CODE>"/*"      {BEGIN(COMMENT);}
<CODE>.|\n      {printf("%s",yytext);}

<COMMENT>"*/"   {BEGIN(CODE);}
<COMMENT>.|\n   {;}
```

Fichero de entrada

```
b=30 + a;
/*
    a=b;
    // fgh
*/
b=c;
```

¿Cuál es la salida?

```
b=30 + a;

b=c;
```

# Ejemplo entornos de reconocimiento

```
%{
      #include<stdio.h>
%}

%s COM COD

%%
                BEGIN(COD);
"//".*\n        printf("%s",yytext);

<COD>"/*"       BEGIN(COM);
<COD>.|\n       printf("%s",yytext);

<COM>"*/"       BEGIN(COD);
<COM>.|\n       ;
```

Con la entrada

```
abc
/*
    cde
    // fgh
*/
ijk
//lmn
```

¿Cuál es la salida?

a)

```
abc
ijk
//lmn
```

b)

```
abc
// fgh
ijk
//lmn
```

c)

```
abc
cde
// fgh
ijk
//lmn
```

INITIAL se reconoce aunque esté dentro de otro entorno

# 06. Compilación

# Compilación en linux

1. Compilamos con lex

**flex** <nombre.l> ---------> genera **lex.yy.c**

Si detecta errores aquí, están en la parte de lex

2. Compilamos con C

**cc –o** <nombre.exe> **lex.yy.c** -------> genera nombre.exe

Aquí detecta los errores de C (en las acciones y/o funciones)

3. Ejecutamos

./nombre.exe